Icare

Manuel et documentation

Contrôleur DMX web pour groupes amateurs

Version 0.2.0 — octobre 2026

En une phrase. Icare transforme un adaptateur USB-DMX (compatible Enttec) en pupitre lumière pilotable depuis le navigateur : couleurs, intensité, effets, séquences et lecture sur scène en BPM.

Auteur : Icare contributors

Dépôt : `https://github.com/DlWAT/Icare`

Licence : MIT

Table des matières

I Guide utilisateur **1**

1 Présentation **2**
1.1 Principes directeurs . . . 2
1.2 Vocabulaire . . . 2
1.3 Les trois onglets . . . 3

2 Installation et démarrage **4**
2.1 Prérequis . . . 4
2.2 Démarrage depuis les sources . . . 4
2.3 Connecter l'adaptateur DMX . . . 4
2.4 Construire un exécutable Windows . . . 4
2.5 Où Icare écrit-il ses fichiers ? . . . 5

3 L'onglet Direct **6**
3.1 Connexion . . . 6
3.2 Couleur et intensité . . . 6
3.3 Position et orientation . . . 6
3.4 Les modes . . . 6
3.5 Visualisation 3D . . . 7

4 L'onglet Séquence **8**
4.1 Fichiers . . . 8
4.2 Barre d'édition . . . 8
4.3 Édition directe du tableau . . . 8

5 L'onglet Live **10**
5.1 Charger et sélectionner . . . 10
5.2 Régler le tempo . . . 10
5.3 Transport . . . 10
5.4 Vue 3D . . . 10

6 Réglages et persistance **11**

II Comprendre Icare **12**

7 Le DMX512 en bref **13**
7.1 Un univers de 512 canaux . . . 13
7.2 Le plan de canaux d'Icare . . . 13

8 L'adaptateur USB-DMX **14**
8.1 La trame Enttec USB Pro . . . 14

8.2 Détection du port 15

9 La chaîne logicielle, de bout en bout 16
9.1 Pourquoi un thread ? 16
9.2 L'état partagé 17

10 Les effets procéduraux 18
10.1 Le strobe en superposition 18

11 Le mode audio-réactif 19

III Documentation technique 20

12 Architecture du projet 21
12.1 Arborescence 21
12.2 La fabrique d'application 21
12.3 Les singletons de service 22
12.4 Configuration 22
12.5 Le front-end 22

13 Le contrôleur DMX 24
13.1 Des modes exclusifs, un overlay 24
13.2 Le thread de sortie 24
13.3 Points de robustesse 25

14 Le lecteur de séquence (LivePlayer) 26
14.1 Chargement 26
14.2 Transport 26
14.3 Boucle de lecture 26
14.4 Sélection de modules et visualisation 27
14.5 Tap-tempo 27

15 Le langage des séquences (DSL) 28
15.1 Grammaire 28
15.2 Structure du code 28
15.3 Le jeton de continuation 28

16 Schémas de fichiers et migration 30
16.1 Format moderne 30
16.2 Format historique 30
16.3 Normalisation 30

IV API, tests et déploiement 32

17 API REST 33
17.1 Conventions 33
17.2 Pages HTML 33
17.3 Onglet Direct 33
17.4 Séquence, Live, palettes, réglages 34

18 Tests, qualité et déploiement 36
18.1 Suite de tests 36

18.2 Qualité du code 36
18.3 Construire l'exécutable 37
18.4 Sources et archive 37

V Des lumières pilotées par Ableton 38

19 La refonte multi-projecteurs 39
19.1 Les profils de projecteurs 39
19.2 Le patch par défaut 39
19.3 Comment le rendu gère les différences 40

20 Les « looks » 41
20.1 Cibles 41
20.2 Résolution 41

21 Le lien avec Ableton Live 42
21.1 Principe 42
21.2 Installation 42
21.3 Mapper les scènes aux looks 43
21.4 Produire son show 43

22 La simulation 3D 44

23 Architecture logicielle mise à jour 45

A Aide-mémoire du DSL 46

B Dépannage 47

C Glossaire 48

Première partie

Guide utilisateur

Chapitre 1

Présentation

ICARE est un petit contrôleur de lumière DMX qui se présente comme une application web locale. Concrètement : un programme Python démarre un serveur *Flask* sur votre machine, et votre navigateur affiche l'interface. Un adaptateur USB-DMX branché en USB assure la liaison physique avec les projecteurs.

Le projet vise les **groupes amateurs** qui veulent se passer d'un vrai pupitre et d'un éclairagiste dédié : on branche, on ouvre la page, et les lumières répondent.

1.1 Principes directeurs

— **100 % local.** Rien ne part sur Internet ; le navigateur ne parle qu'à `127.0.0.1:5000`. Pas de compte, pas de cloud.
— **Une seule interface.** Le navigateur est le pupitre : trois onglets suffisent (*Direct*, *Séquence*, *Live*).
— **Démarrage rapide.** Une commande pour installer, une commande pour lancer. Une version `.exe` autonome existe aussi (voir chapitre 2).
— **Le spectaculaire accessible.** Effets intégrés (arc-en-ciel, feu, gyrophare, chenillard), mode *audio-réactif* au microphone, et lecture calée au tempo avec le bouton *TAP*.

1.2 Vocabulaire

Avant d'aller plus loin, fixons les mots employés dans ce document et dans l'interface.

Univers (DMX)
Ensemble des 512 canaux d'une ligne DMX. Chaque canal porte une valeur de 0 à 255.

Module / projecteur (*fixture*)
Un appareil branché sur la ligne, qui occupe un ou plusieurs canaux. Dans ICARE, les modules portent des noms comme `ParLED1` ou `Strobe1`.

Canal de base
Premier canal d'un projecteur. Si un PAR RGB démarre au canal 9, alors R, G et B occupent respectivement les canaux 9, 10 et 11.

Séquence
Une suite d'étapes (*lignes*) décrivant l'état des projecteurs, enregistrée dans un fichier JSON.

Section
Étiquette donnée à un groupe de lignes (`Intro_x4`, `Couplet1`…) pour se repérer.

Beat / ligne
Une unité de temps de la séquence. Une ligne = un temps musical.

BPM
Beats per minute. Cadence de lecture de la séquence Live.

1.3 Les trois onglets

La barre de navigation en haut de page donne accès aux trois espaces de travail, résumés dans le tableau 1.1.

Onglet	À quoi il sert
Direct	Pilotage immédiat : couleurs, intensité, pan/tilt, blackout, couleurs aléatoires, strobe, effets intégrés, audio-réactif et visualisation 3D.
Séquence	Édition d'un show temps par temps : table de lignes par module, insertion d'effets *Uniform*, *Fondu* et *Strobe*, sauvegarde dans un fichier JSON.
Live	Lecture d'une séquence sur scène : choix des modules, BPM réglable ou mesuré au *TAP*, transport, barre de progression et vue 3D du faisceau.

TABLE 1.1 – Rôle des trois onglets.

En complément, la **barre supérieure** (ou *topbar*) affiche un témoin d'état DMX et un bouton de bascule de thème clair/sombre.

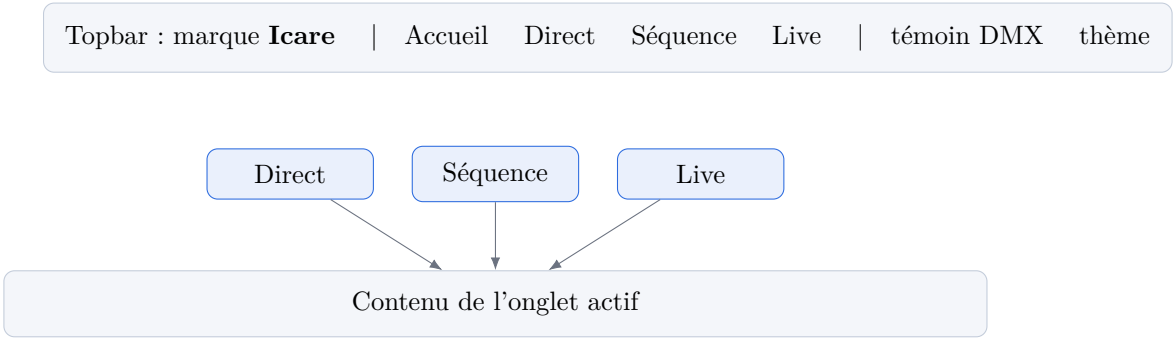


FIGURE 1.1 – Organisation générale de l'interface.

Chapitre 2

Installation et démarrage

2.1 Prérequis

— Windows (les autres systèmes fonctionnent en développement, mais le paquetage fourni cible Windows).
— Python 3.10 ou plus récent.
— Un adaptateur USB-DMX compatible *Enttec USB Pro* (ou un clone qui implémente le même protocole).

2.2 Démarrage depuis les sources

```
python -m venv .venv
.\.venv\Scripts\Activate.ps1
pip install -r requirements.txt
python run_webapp.py
```

Listing 2.1 – Installation et lancement en développement.

Le serveur écoute sur `http://127.0.0.1:5000`. Ouvrez cette adresse dans votre navigateur. En mode *exécutable* empaqueté, le navigateur s'ouvre automatiquement au démarrage.

2.3 Connecter l'adaptateur DMX

1. Branchez l'adaptateur USB-DMX sur le PC et reliez-le à la ligne DMX des projecteurs (XLR 5 broches ou 3 broches selon votre câblage).
2. Dans l'onglet **Direct**, renseignez le *canal de base* du premier projecteur (9 par défaut), puis cliquez sur **Connexion DMX**.
3. Le témoin en haut à droite passe au vert (*DMX OK*).

Si aucun port n'est indiqué, Icare choisit automatiquement le premier port COM dont la description contient « USB ». Vous pouvez forcer un port en le saisissant dans le champ *Port* (par exemple `COM3`).

2.4 Construire un exécutable Windows

Pour distribuer Icare sur un PC sans Python, utilisez le script fourni :

```
.\build_exe.ps1
```

Listing 2.2 – Construction de l'exécutable autonome.

Cela produit `dist\IcareWeb.exe` (environ 30 Mo). Au lancement, l'exécutable démarre le serveur, ouvre le navigateur et crée ses fichiers de travail à côté de lui.

2.5 Où Icare écrit-il ses fichiers ?

Fichier / dossier	Type	Contenu
`Sequences/`	dossiers	Shows en JSON (fournis + créés).
`Palettes/`	dossier	Palettes de couleurs nommées.
`icare_settings.json`	fichier	Préférences persistantes (thème, port, canal de base, dernière séquence, favoris).
`icare.log`	fichier	Journal rotatif (512 Ko × 3).

TABLE 2.1 – Fichiers produits à l'exécution.

En développement, ces fichiers apparaissent à la racine du projet. En version empaquetée, ils sont écrits à côté de l'exécutable, jamais dans l'archive lecture seule.

Chapitre 3

L'onglet Direct

L'onglet **Direct** est le pupitre manuel : tout ce qu'on y fait est envoyé immédiatement au contrôleur DMX.

3.1 Connexion

Le panneau *Connexion* contient :

— **Canal de base (R)** : premier canal couleur du projecteur (0 à 509).
— **Port (optionnel)** : nom du port COM ; vide = auto-détection.
— **Connexion DMX** : ouvre le port série.
— **Mettre à jour canal de base** : change le canal de base sans rouvrir le port.

3.2 Couleur et intensité

Les curseurs **Rouge**, **Vert** et **Bleu** vont de 0 à 255 ; le curseur **Amplitude** règle la luminosité globale. Le sélecteur de couleur (color picker) pilote directement les trois canaux RGB. Un aperçu rectangulaire montre la couleur résultante.

Le bouton + **Favori** enregistre la couleur courante dans la palette de favoris ; un clic sur une pastille la rappelle. La croix `x` retire un favori. La liste est persistée dans les réglages (32 favoris maximum).

3.3 Position et orientation

Les curseurs **Pan** et **Tilt** pilotent les canaux DMX 3 et 5. Les quatre réglages suivants (*Angle du faisceau*, *Position X*, *Position Z*, *Hauteur Y*) ne sont **pas** envoyés au DMX : ils paramètrent uniquement la visualisation 3D, pour se représenter la scène.

3.4 Les modes

Blackout / Stop. **Blackout** éteint tout immédiatement (couleurs à 0, amplitude à 0). **Stop** arrête le mode en cours.

Couleurs aléatoires. Trois paramètres : *fréquence* (Hz), *lissage* (0–100 %) et *luminosité* (0–255). Le lissage interpole progressivement vers la couleur cible pour un rendu plus doux.

Strobe (overlay). Un strobe qui *se superpose* au mode courant sans l'interrompre. Réglages : fréquence (Hz), rapport cyclique (%), canal DMX optionnel et durée (s). Si un canal est précisé, c'est ce canal qui clignote ; sinon l'image entière est alternée entre la couleur courante et le noir.

Bibliothèque d'effets. Quatre effets procéduraux : **Rainbow**, **Fire**, **Police** et **Chase RGB**, avec réglage de *vitesse* et de *luminosité*. Voir le chapitre 10 pour le détail.

Audio-réactif (bêta). Le microphone du navigateur pilote l'intensité au rythme de la musique. On règle la *sensibilité* et la *couleur* ; une barre indique le niveau détecté. Voir le chapitre 11.

3.5 Visualisation 3D

Le panneau *Visualisation 3D* dessine un projecteur, son faisceau et un faisceau spot sur une scène. Couleur, intensité et orientation suivent les curseurs (et les modes actifs). Pendant un strobe overlay, l'opacité du faisceau clignote au même rythme.

Chapitre 4

L'onglet Séquence

L'onglet **Séquence** est l'éditeur de spectacle. Il manipule un fichier JSON que l'on pourra lire ensuite dans l'onglet *Live*.

4.1 Fichiers

Le panneau *Fichiers* liste les séquences disponibles. Chaque entrée offre un bouton **Charger** et une croix `x` de suppression. Le champ du haut permet de nommer puis **Sauvegarder** le show courant.

4.2 Barre d'édition

Champ	Signification
Module	Projecteur ciblé par l'insertion d'effet.
#Ligne	Numéro de la ligne (temps) où insérer.
#Tps	Nombre de temps que dure l'effet (`n`).
Fréquence	Fréquence du strobe, en Hz.
Couleur 1 / Couleur 2	Couleurs de départ et d'arrivée.

Table 4.1 – Contrôles de la barre d'édition.

Trois boutons insèrent des effets :

— **Uniform** : couleur fixe pendant `n` temps.
— **Fondu** : dégradé linéaire de la couleur 1 vers la couleur 2 sur `n` temps.
— **Strobe** : clignotement à la fréquence donnée pendant `n` temps.

Le bouton **+ Ligne** ajoute une ligne vierge, et **Effacer cellule** vide la cellule ciblée.

4.3 Édition directe du tableau

Le tableau comporte une colonne par module. Les cellules de fonction sont éditionnables directement : on peut y taper à la main un jeton du langage décrit au chapitre 15. Le caractère `|` prolonge l'effet commencé sur la ligne précédente.

À noter

Une cellule vide signifie « ne rien changer pour ce module à ce temps ». C'est utile pour ne modifier qu'un seul projecteur à la fois.

Chapitre 5

L'onglet Live

L'onglet **Live** joue une séquence en conditions de spectacle.

5.1 Charger et sélectionner

1. Choisissez une séquence dans la liste (**Charger**).
2. Cochez les **modules** à piloter. Par défaut, tous les modules de la séquence sont pris en compte. Les boutons *Tout sélectionner* et *Tout désélectionner* font le tri.

5.2 Régler le tempo

Deux méthodes :

— Saisir un **BPM** (20 à 300) dans le champ numérique.
— Cliquer sur **TAP** en rythme : après deux appuis rapprochés, ICARE calcule le tempo moyen et l'applique. **Reset** efface l'historique des appuis.

5.3 Transport

Play, **Pause**, **Stop**, **Précédent** et **Suivant** contrôlent la lecture. La barre de progression, le nom de la section courante et le compteur `i / total` donnent l'avancement. À la fin, la dernière image est maintenue puis la lecture s'arrête.

5.4 Vue 3D

Le faisceau affiché prend la couleur envoyée au DMX en temps réel, ce qui permet de vérifier ce qui part sur la ligne même sans regarder les projecteurs.

Chapitre 6

Réglages et persistance

Peu de choses à régler, mais elles survivent aux redémarrages.

Réglage	Effet
Thème	Clair ou sombre (bouton lune/soleil de la topbar).
Dernier port COM	Pré-remplit le champ port.
Canal de base	Restauré au démarrage.
Dernière séquence	Mémorisée pour rechargement.
Couleurs favorites	Palette affichée dans l'onglet Direct.

TABLE 6.1 – Réglages persistés.

Tout est écrit dans `icare_settings.json`. Le journal `icare.log` enregistre les événements (connexion, erreurs série, etc.) et est automatiquement limité en taille.

Attention

Le fichier de réglages est du JSON : s'il devient illisible (édition manuelle ratée), ICARE le réinitialise silencieusement au démarrage suivant.

Deuxième partie

Comprendre Icare

Chapitre 7

Le DMX512 en bref

Cette partie explique *pourquoi* le code fait ce qu'il fait. On y revoit le protocole DMX, la trame de l'adaptateur, la circulation des données et les algorithmes des effets.

7.1 Un univers de 512 canaux

DMX512 est un protocole de scène très simple. Un *univers* contient **512 canaux**, chacun sur un octet, donc une valeur de **0 à 255**. Onze bits sont envoyés par canal : une impulsion de *start bit*, huit bits de données, deux *stop bits*. Le tout circule sur un bus RS-485 à **250 000 bauds**.

Deux propriétés importantes :

— **Aucun retour.** Le contrôleur parle, les projecteurs écoutent. On ne sait jamais si un projecteur a compris autre chose que ce qu'on voulait.
— **Image continue.** La même *image* (les 512 valeurs) est renvoyée en boucle. Un projecteur qui cesse de recevoir se met en sécurité (souvent après quelques dixièmes de seconde).

Les valeurs n'ont pas de sens universel : c'est le projecteur qui décide, selon son *profil*. Un PAR RGB courant utilise trois canaux (*intensité*, *rouge*, *vert*, *bleu* sur certains modèles, parfois plus).

7.2 Le plan de canaux d'Icare

Pour rester simple, Icare pilote un unique projecteur RGB selon un plan de canaux fixe, hérité de l'ancien prototype. Ces constantes vivent dans `webapp/config.py`, classe `Channels`.

Canal	Constante	Rôle
0	`MASTER`	Toujours à 255 pour «ouvrir» la sortie.
1	`AMPLITUDE`	Luminosité globale (0–255).
3	`PAN`	Orientation horizontale.
5	`TILT`	Orientation verticale.
8	`SHUTTER`	Toujours à 0 (pas d'effet de shutter).
9	`BASE_COLOR_DEFAULT`	R du projecteur, G en 10, B en 11.

Table 7.1 – Plan de canaux utilisé par le contrôleur.

Le *canal de base* est modifiable : les canaux R, G, B sont alors `base`, `base+1` et `base+2`. Icare accepte une base de 0 à 509, afin qu'il reste toujours trois canaux disponibles.

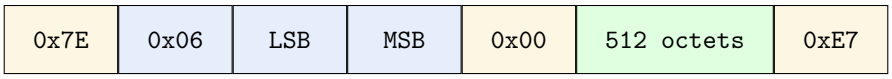
Chapitre 8

L'adaptateur USB-DMX

Le PC n'a pas de port DMX natif : on passe par un boîtier USB dont l'un des plus répandus est l'*Enttec USB Pro*. Le dialogue se fait à 250 000 bauds, en 8 bits, sans parité, 2 bits de stop (8N2). ICARE occupe la classe `DMX_Icare` de `pydmxIcare.py`.

8.1 La trame Enttec USB Pro

Chaque envoi est encapsulé dans une petite trame :



Trame Enttec USB Pro : début, étiquette «Output Only Send DMX Packet Request», longueur de charge utile en petit-boutiste, code de départ DMX, données, fin.

Détail des champs :

— `0x7E` : octet de début de trame (*START_BYTE*).
— `0x06` : étiquette «envoi seul de paquet DMX» (*LABEL_DMX_TX*).
— **LSB, MSB** : longueur de la charge utile = 513 octets (1 code de départ + 512 canaux), soit `0x01 0x02`.
— `0x00` : code de départ DMX standard.
— **512 octets** : les valeurs des canaux.
— `0xE7` : octet de fin de trame (*END_BYTE*).

Dans le code, l'en-tête est précalculé une fois pour toutes sous forme de tableau NumPy, puis concaténé aux 512 canaux à chaque envoi :

```
payload_len = NUM_CHANNELS + 1  # 1 start code + 512 channels
self.start_byte = np.array([
    START_BYTE, LABEL_DMX_TX,
    payload_len & 0xFF, (payload_len >> 8) & 0xFF,
    0x00,  # DMX start code
], np.uint8)
...
frame = np.concatenate((self.start_byte, self.data, self.end_byte)).tobytes()
self.com.write(frame)
```

Listing 8.1 – Construction et envoi d'une trame dans `pydmxIcare.py`.

8.2 Détection du port

Si aucun port n'est fourni, ICARE parcourt les ports série et retient le premier dont la description contient « USB » :

```
for port_no, description, _address in serial.tools.list_ports.comports():
    if "USB" in description:
        return str(port_no)
return None
```

Listing 8.2 – Auto-détection du port série.

En cas d'échec d'écriture, le port est refermé proprement plutôt que de laisser la boucle échouer en rafale.

Chapitre 9

La chaîne logicielle, de bout en bout

Voici comment un mouvement de curseur finit en photons.

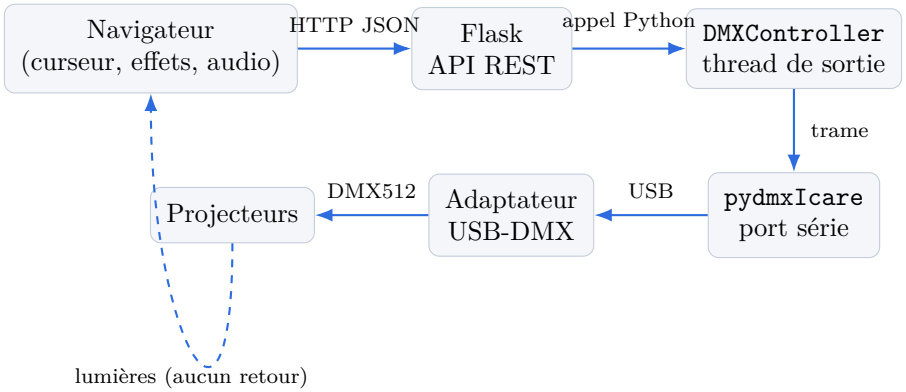


FIGURE 9.1 – Chaîne complète, du navigateur aux projecteurs.

Les étapes :

1. Le navigateur (`app.js`) envoie par exemple `POST /api/experiment/set-values` avec `{r, g, b, amp, pan, tilt}`.
2. La route correspondante (`webapp/api_experiment.py`) appelle `dmx.set_values(...)`.
3. `DMXController` borne les valeurs (0–255) et, selon le mode actif, écrit la *frame* courante.
4. La boucle de fond (thread `dmx-out`) rafraîchit la sortie en continu (environ 20 Hz en mode stable).
5. `pydmxIcare` assemble la trame Enttec et l'écrit sur le port série.
6. L'adaptateur traduit cela en signal DMX512 sur la ligne.

9.1 Pourquoi un thread ?

L'écriture série et les effets sont des tâches *continues* qui ne doivent pas bloquer les requêtes HTTP. ICARE isole donc la sortie DMX dans un thread démon, et le lecteur de séquence dans un second thread. Les routes HTTP ne font que modifier l'état partagé (couleurs, paramètres de mode) ; le thread s'occupe de le refléter sur le fil.

9.2 L'état partagé

Le contrôleur maintient :

— `r, g, b, amp, pan, tilt` : les valeurs de sortie ;

— `enabled, strobe_enabled, random_enabled, effect_enabled, audio_enabled` : un seul mode actif à la fois ;

— `overlay` : le strobe superposé, indépendant du mode ;

— `random_params, effect_params, base_color_channel`.

Chapitre 10

Les effets procéduraux

Quatre effets sont générés côté serveur, dans la boucle de sortie. Le temps écoulé depuis le démarrage, multiplié par la *vitesse*, sert de paramètre d'animation.

Effet	Principe
`rainbow`	Teinte qui tourne : $h = (60\,t) \bmod 360$ degrés, saturation et valeur maximales (conversion HSV→RGB).
`fire`	Orange/rouge vacillant : teinte autour de 18°, valeur aléatoire entre 0,55 et 1,0 pour l'effet de flamme.
`police`	Alternance rouge / bleu à environ 4 Hz ($\lfloor 4t \rfloor \bmod 2$).
`chase`	Chenillard qui parcourt R, G, B à environ 3 pas par seconde.

Table 10.1 – Effets procéduraux intégrés.

La conversion HSV vers RGB est implémentée à la main, sans dépendance externe :

```
c = v * s
x = c * (1 - abs(((h / 60) % 2) - 1))
m = v - c
if   h <  60: r1, g1, b1 = c, x, 0
elif h < 120: r1, g1, b1 = x, c, 0
...
return int((r1 + m) * 255), int((g1 + m) * 255), int((b1 + m) * 255)
```

Listing 10.1 – Le cœur de `_hsv_to_rgb`.

10.1 Le strobe en superposition

Le strobe d'overlay est à part : il ne remplace pas le mode courant, il passe par-dessus. À chaque image, on évalue une phase $\phi = (t \cdot f) \bmod 1$ et on éteint la sortie quand ϕ dépasse le rapport cyclique :

$$\text{sortie} = \begin{cases} \text{couleur courante} & \text{si } \phi < \text{duty}, \\ \text{noir (ou 0 sur le canal choisi)} & \text{sinon.} \end{cases}$$

Si un canal DMX est précisé, seul ce canal clignote ; sinon toute l'image alterne. L'overlay s'arrête tout seul à la fin de la durée demandée.

Chapitre 11

Le mode audio-réactif

Ce mode transforme le microphone du navigateur en capteur de rythme. Toute l'analyse se fait côté client ; le serveur ne reçoit que le résultat.

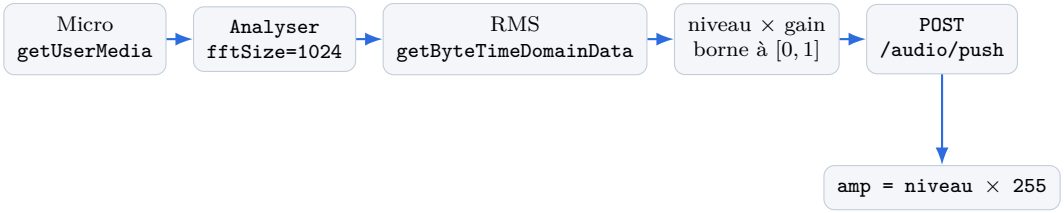


FIGURE 11.1 – Chaîne audio-réactive.

Du son au niveau. On échantillonne la forme d'onde, on recentre les octets autour de 128, puis on calcule la valeur quadratique moyenne (RMS) :

$$\mathrm{RMS} = \sqrt{\frac{1}{N}\sum_{i=0}^{N-1}\left(\frac{d_i - 128}{128}\right)^2}, \qquad \text{niveau} = \min\bigl(1,\ \mathrm{RMS} \times \text{gain}\bigr).$$

Le curseur *Sensibilité* règle le `gain` entre 0,5 et 6.

Fréquence d'envoi. La boucle d'animation tourne à la fréquence de l'écran, mais l'envoi réseau est limité à environ 30 Hz pour ne pas saturer le serveur. La couleur choisie accompagne le niveau.

Côté serveur. `audio_push(niveau, couleur)` met à jour l'amplitude à `niveau` × `255`, applique éventuellement la couleur, et renvoie l'image. Le mode ne génère rien tout seul : sans échantillons, l'état ne change pas.

Troisième partie

Documentation technique

Chapitre 12

Architecture du projet

12.1 Arborescence

```
Icare/
|-- pydmxIcare.py          # bas niveau : trame Enttec + port serie
|-- pydmxlight.py          # descripteurs de projecteurs (Par LED, Strobe)
|-- run_webapp.py          # point d'entree (python run_webapp.py)
|-- webapp/
|   |-- __init__.py        # fabrique create_app()
|   |-- config.py          # constantes, UserSettings, journalisation
|   |-- services.py        # singletons (settings, dmx)
|   |-- utils.py           # helpers partages (noms de fichiers surs)
|   |-- dmx_controller.py  # machine a etats + thread de sortie DMX
|   |-- live_player.py     # lecteur de sequence + tap-tempo
|   |-- dsl.py             # analyseur du langage de sequences
|   |-- schemas.py         # validation / migration des fichiers JSON
|   |-- pages.py           # routes HTML
|   |-- api_experiment.py  # API onglet Direct
|   |-- api_sequence.py    # API editeur de sequence
|   |-- api_live.py        # API lecture Live
|   |-- api_health.py      # /api/health
|   |-- api_palettes.py    # favoris + palettes nommees
|   |-- api_settings.py    # preferences
|   |-- templates/         # gabarits Jinja2
|   `-- static/            # app.css, app.js, vendor/three.min.js
|-- Sequences/             # shows d'exemple
|-- tests/                 # suite pytest
|-- docs/                  # cette documentation (LaTeX)
|-- pyproject.toml         # build, ruff, pytest
`-- build_exe.ps1          # paquetage PyInstaller
```

Listing 12.1 – Arborescence du dépôt.

12.2 La fabrique d'application

`create_app()` construit et configure l'application Flask. Elle choisit le dossier des gabarits et des fichiers statiques selon qu'on tourne depuis les sources ou depuis un exécutable empaqueté (`sys._MEIPASS`), puis enregistre les *blueprints* un par un :

```
from .pages import pages_bp
from .api_experiment import api_experiment_bp
...
```

```
app.register_blueprint(pages_bp)
app.register_blueprint(api_experiment_bp)
app.register_blueprint(api_sequence_bp)
app.register_blueprint(api_live_bp)
app.register_blueprint(api_health_bp)
app.register_blueprint(api_palettes_bp)
app.register_blueprint(api_settings_bp)
```

Listing 12.2 – Enregistrement des blueprints (`webapp/__init__.py`).

Chaque famille de routes vit dans son propre module : on retrouve donc naturellement une route `/api/experiment/...` dans `api_experiment.py`, et ainsi de suite.

12.3 Les singletons de service

Pour éviter les imports circulaires et garantir une seule instance partagée, `webapp/services.py` expose :

— `settings` : les préférences chargées depuis le disque ;
— `dmx` : l'unique `DMXController` du processus ;
— `SEQ_DIR` / `PALETTE_DIR` : les chemins des dossiers de données.

À noter
Il n'existe qu'**un** contrôleur DMX et **un** lecteur Live pour tout le serveur. C'est voulu : un seul fil pilote la ligne DMX.

12.4 Configuration

`webapp/config.py` distingue deux choses :

— les **constantes** : plan de canaux (`Channels`) et cadences de boucle (`Timings`) ;
— l'**état persistant** : la dataclass `UserSettings`.

`UserSettings` sait se charger, se sauvegarder et mettre à jour des champs. Au chargement, seules les clés connues sont conservées, ce qui rend le fichier tolérant aux évolutions de schéma. La journalisation écrit à la fois sur la console et dans un fichier rotatif.

Cadence	Valeur	Rôle
`STEADY_LOOP`	0,05 s	Sortie stable, environ 20 Hz.
`RANDOM_LOOP`	0,025 s	Environ 40 Hz pour des fondus fluides.
`EFFECT_LOOP`	0,025 s	Environ 40 Hz pour les effets.
`IDLE_LOOP`	0,10 s	Attente quand aucun mode n'est actif.

Table 12.1 – Cadences de la boucle DMX (`Timings`).

12.5 Le front-end

`webapp/static/app.js` est un unique script sans dépendance (hormis Three.js, fourni en local dans `vendor/`). Au chargement, il lit `document.body.dataset.page` et n'active que le module correspondant (`bindExperiment`, `bindSequence` ou `bindLive`). Deux sondages réguliers maintiennent l'interface synchronisée avec l'état serveur :

— `GET /api/experiment/status` toutes les 200 ms (onglet Direct) ;
— `GET /api/live/status` toutes les 250 ms (onglet Live).

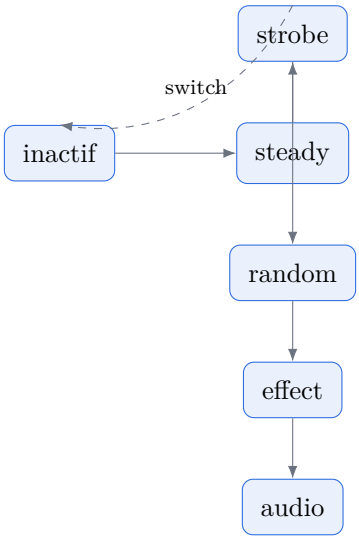
Chapitre 13

Le contrôleur DMX

`DMXController` (`webapp/dmx_controller.py`) est le cœur temps réel.

13.1 Des modes exclusifs, un overlay

Un seul mode peut être actif à la fois. Le strobe d'overlay, lui, est indépendant et se superpose.



`_switch_mode()` réinitialise tous les drapeaux puis active le mode voulu : les modes sont mutuellement exclusifs.

13.2 Le thread de sortie

Le thread `dmx-out` boucle sans fin et exécute une étape selon le mode actif :

```
while not self._stop_event.is_set():
    try:
        if self.enabled:          self._send_with_overlay(); time.sleep(Timings.
    STEADY_LOOP)
        elif self.strobe_enabled: self._step_strobe()
        elif self.random_enabled: self._step_random()
        elif self.effect_enabled: self._step_effect()
        elif self.audio_enabled:  time.sleep(Timings.IDLE_LOOP)
        else:                     time.sleep(Timings.IDLE_LOOP)
    except Exception:
```

```
10          logger.exception("DMX loop error; pausing 0.5 s")
11          time.sleep(0.5)
```

Listing 13.1 – Boucle principale du contrôleur (simplifiée).

Chaque *frame* passe par `_send_with_overlay()`, qui applique d'abord l'overlay éventuel puis envoie l'image. Les écritures série sont encapsulées dans des `try/except` : une erreur transitoire est journalisée et la boucle repart après une courte pause, au lieu de tuer le thread.

13.3 Points de robustesse

— Toutes les entrées sont **bornées** : les composantes RGB, l'amplitude, le pan et le tilt sont contraints à 0–255.
— Le canal de base est validé (0–509) et refusé sinon.
— `pydmxIcare.send_data()` ne fait rien si le port est fermé : on peut lancer l'application sans matériel, sans plantage.
— En cas d'erreur d'écriture série, le port est refermé pour permettre une reconnexion propre.

Chapitre 14

Le lecteur de séquence (LivePlayer)

`LivePlayer` (`webapp/live_player.py`) parcourt les lignes d'une séquence et traduit chaque cellule en couleurs.

14.1 Chargement

`load(raw)` normalise la séquence (voir chapitre 16) puis remet l'index et l'état de lecture à zéro. Aucune lecture ne démarre tant que `start()` n'est pas appelé.

14.2 Transport

Méthode	Effet
`start(bpm)`	Démarre la lecture (BPM optionnel).
`pause()`	Bascule l'état *pause*.
`stop()`	Arrête et revient au début.
`next()` / `prev()`	Avance ou recule d'une ligne.
`toggle_module(name)`	(Dé)sélectionne un module.
`select_all()` / `unselect_all()`	Sélection globale.

Table 14.1 – API de transport du lecteur.

14.3 Boucle de lecture

Le thread `live-player` applique une ligne puis attend la durée d'un temps : `60 / BPM` secondes. À la fin de la séquence, la dernière image est maintenue avant l'arrêt.

```
self.index = min(self.index + 1, total)
line = self.data["data"].get(str(self.index), {})
self._apply_step(line.get("function", []))
time.sleep(60.0 / max(1, self.bpm))
if self.index >= total:
    self.playing = False
```

Listing 14.1 – Cœur de la boucle de lecture.

14.4 Sélection de modules et visualisation

Si l'utilisateur a coché des modules, seuls ceux-là sont appliqués ; sinon tous le sont. Pour la vue 3D, le lecteur retient la couleur du *premier* module ciblé et la pousse au contrôleur DMX, qui est mono-projecteur.

14.5 Tap-tempo

Le tempo mesuré vit dans `live_player.py`. On conserve les horodatages récents (fenêtre de 3 secondes), on calcule les intervalles, on en fait la moyenne, et on borne le résultat entre 20 et 300 BPM :

```
intervals = [b - a for a, b in zip(self._taps, self._taps[1:], strict=False)]
avg = sum(intervals) / len(intervals)
bpm = int(round(60 / avg))
bpm = max(20, min(300, bpm))
```

Listing 14.2 – Calcul du tempo à partir des appuis.

Chapitre 15

Le langage des séquences (DSL)

Chaque cellule d'une ligne contient une chaîne qui décrit l'action d'**un** projecteur pour ce temps. `webapp/dsl.py` la transforme en objet typé.

15.1 Grammaire

Jeton	Signification
`""`	Ne rien changer (garder l'état courant).
`|`	Continuer l'effet multi-temps précédent.
`uni_n_R_G_B`	Couleur fixe pendant `n` temps.
`fond_n_R1_G1_B1_R2_G2_B2`	Fondu linéaire de c_1 vers c_2.
`str_n_R_G_B_freq`	Strobe à `freq` Hz pendant `n` temps.

TABLE 15.1 – Jetons du langage de séquences.

La fréquence du strobe est optionnelle : sans elle, la valeur par défaut est 60 Hz. Les longueurs `n` inférieures à 1 sont ramenées à 1, et toute composante hors de 0–255 lève une `DSLError`.

15.2 Structure du code

Le module expose des dataclasses gelées :

— `UniformEffect(length, color)` ;
— `FadeEffect(length, color_from, color_to)` ;
— `StrobeEffect(length, color, freq)` ;
— `ContinueEffect()` pour le jeton `|`.

La fonction `parse(token)` renvoie l'objet correspondant, ou `None` pour une cellule vide. Trois *encodeurs* (`encode_uniform`, `encode_fade`, `encode_strobe`) font l'opération inverse, utilisés notamment par la migration d'anciens fichiers.

15.3 Le jeton de continuation

Un effet de `n` temps s'écrit sur une première ligne, puis `n-1` fois `|`. Le lecteur conserve, pour chaque module, un petit état (mode, longueur, pas courant, couleurs, fréquence). À chaque `|`, il avance d'un pas :

— **fondu** : interpolation linéaire $c = c_1 + \frac{\text{pas}}{n-1}\,(c_2 - c_1)$;
— **strobe** : alternance couleur / noir à chaque pas ;
— **couleur fixe** : la couleur reste identique.

```
fond_4_255_0_0_0_0_255   |   |   |
```

Listing 15.1 – Exemple : un fondu de 4 temps du rouge au bleu.

Chapitre 16

Schémas de fichiers et migration

Deux formats de séquence sont acceptés. `webapp/schemas.py` les unifie en un seul : le format *moderne*.

16.1 Format moderne

```json
{
  "Header": {
    "Structure": ["Intro_x4", "Intro_x4", "Couplet1_A_x2"],
    "Tempo": "60",
    "Modules": ["ParLED1", "ParLED2", "Strobe1"],
    "Line": ["1", "2", "3"]
  },
  "data": {
    "1": { "section": "Intro_x4", "function": ["", "uni_4_78_243_86", ""] },
    "2": { "section": "Intro_x4", "function": ["", "|", "str_3_132_214_107_63"] }
  }
}
```

Listing 16.1 – Séquence au format moderne.

16.2 Format historique

Les anciens fichiers (comme les premières versions de `Doctor_doctor.json`) ont une autre forme : une clé `headers` avec une `structure`, et des lignes de quatre nombres `[durée_beats, R, G, B]`.

```json
{
  "headers": { "structure": ["intro", "Couplet1", "Refrain1"] },
  "data": { "intro": [[2, 125, 20, 35], [2, 20, 53, 128]] }
}
```

Listing 16.2 – Ancien format (migré automatiquement).

16.3 Normalisation

`normalize_sequence(raw)` renvoie toujours le format moderne :

— les modules manquants sur une ligne sont complétés par des cellules vides, ou tronqués s'il y en a trop ;
— les clés de lignes sont converties en chaînes `"1"`, `"2"`, … et triées numériquement ;

— en format historique, chaque durée est multipliée par deux (une durée de 0,5 donne un temps, une durée de 2 en donne quatre) ; la première ligne reçoit un jeton `uni_...`, les suivantes `|` ;
— un module factice `ParLED1` est utilisé, l'ancien format ne distinguant pas les projecteurs.

Si `raw` n'a ni `Header` ni `headers`, une `ValueError` est levée : c'est ce qui alimente les réponses d'erreur 422 de l'API.

À noter

À la sauvegarde, l'API valide en faisant un aller-retour de normalisation avant d'écrire le fichier : un contenu invalide est donc refusé au lieu d'être persisté.

Quatrième partie

API, tests et déploiement

Chapitre 17

API REST

L'interface et le serveur communiquent exclusivement en JSON.

17.1 Conventions

— **Succès** : `{"ok": true, ...}`, code HTTP `200`.
— **Échec** : `{"ok": false, "error": "..."}`, avec un code explicite :
  — `400` : requête mal formée ou paramètre invalide ;
  — `404` : fichier introuvable ;
  — `422` : contenu illisible ou schéma inconnu ;
  — `500` : erreur d'entrée/sortie côté serveur ;
  — `503` : connexion DMX impossible.
— Les noms de fichiers envoyés sont nettoyés et confinés dans `Sequences/` ou `Palettes/` : aucune traversée de répertoire n'est possible.

17.2 Pages HTML

Méthode	Route	Page
GET	`/`	Accueil
GET	`/experiment`	Onglet Direct
GET	`/sequence`	Onglet Séquence
GET	`/live`	Onglet Live

Table 17.1 – Routes HTML.

17.3 Onglet Direct

Méthode	Route	Rôle
POST	`/api/experiment/connect`	Ouvre le port (canal de base + port).
POST	`/api/experiment/base-channel`	Change le canal de base.
POST	`/api/experiment/set-values`	RGB, amplitude, pan, tilt.
POST	`/api/experiment/start`	Démarre la sortie stable.
POST	`/api/experiment/stop`	Arrête le mode courant.
POST	`/api/experiment/blackout`	Éteint tout.

Méthode	Route	Rôle
POST	`/api/experiment/strobe`	Strobe plein cadre (freq, shuffle).
POST	`/api/experiment/random`	Configure et démarre l'aléatoire.
POST	`/api/experiment/mode/random-config`	Aléatoire : paramètres seuls.
POST	`/api/experiment/random-start`	Démarre l'aléatoire.
POST	`/api/experiment/random-stop`	Arrête l'aléatoire.
POST	`/api/experiment/mode/strobe-overlay`	Strobe superposé.
POST	`/api/experiment/effect/start`	Lance un effet (rainbow, ...).
POST	`/api/experiment/effect/stop`	Arrête l'effet.
POST	`/api/experiment/audio/start`	Active le mode audio.
POST	`/api/experiment/audio/stop`	Désactive le mode audio.
POST	`/api/experiment/audio/push`	Pousse un échantillon (niveau, couleur).
GET	`/api/experiment/status`	État complet (polling 200 ms).

TABLE 17.2: Points d'entrée de l'onglet Direct.

Exemple de corps pour `set-values` :

```
{ "r": 255, "g": 128, "b": 0, "amp": 200, "pan": 90, "tilt": 45 }
```

17.4 Séquence, Live, palettes, réglages

Méthode	Route	Rôle
Séquence		
GET	`/api/sequence/list`	Liste les fichiers JSON.
GET	`/api/sequence/load?name=`	Charge et normalise.
POST	`/api/sequence/save`	Sauve (validation par aller-retour).
POST	`/api/sequence/delete`	Supprime un fichier.
Live		
GET	`/api/live/list`	Liste les séquences lisibles.
POST	`/api/live/load`	Charge une séquence dans le lecteur.
POST	`/api/live/play`	Démarre (BPM optionnel).
POST	`/api/live/pause`	Bascule pause.
POST	`/api/live/stop`	Arrête et rembobine.
POST	`/api/live/next` / `prev`	Ligne suivante / précédente.
POST	`/api/live/toggle-module`	(Dé)sélectionne un module.
POST	`/api/live/select-all` / `unselect-all`	Sélection globale.
POST	`/api/live/tap`	Enregistre un appui de tempo.
POST	`/api/live/tap-reset`	Efface l'historique des appuis.
POST	`/api/live/bpm`	Fixe le BPM (20–300).
GET	`/api/live/status`	Progression, section, BPM, couleur.
Palettes		
GET	`/api/palettes/favorites`	Liste des couleurs favorites.
POST	`/api/palettes/favorites/add`	Ajoute un favori.
POST	`/api/palettes/favorites/remove`	Retire un favori.
GET	`/api/palettes/list`	Liste les palettes nommées.
GET	`/api/palettes/load?name=`	Charge une palette.
POST	`/api/palettes/save`	Enregistre une palette.

Méthode	Route	Rôle
POST	`/api/palettes/delete`	Supprime une palette.
Divers		
GET	`/api/settings`	Réglages courants.
POST	`/api/settings`	Met à jour (thème, port, …).
GET	`/api/health`	État du service et du DMX.

TABLE 17.3: Points d'entrée restants.

Chapitre 18

Tests, qualité et déploiement

18.1 Suite de tests

Les tests utilisent `pytest` et ne nécessitent **aucun matériel** : le port série est simplement fermé, donc les envois sont sans effet.

```
pip install -r requirements-dev.txt
pytest
```

Listing 18.1 – Lancer la suite.

Fichier	Ce qui est vérifié	Tests
`test_api.py`	Routes HTTP, bornage, sécurité des noms, favoris, réglages.	10
`test_dmx_controller.py`	Bornage RGB/amp/pan/tilt, canal de base, HSV, effets inconnus.	7
`test_dsl.py`	Analyse, erreurs, bornes, aller-retours des encodeurs.	16
`test_schemas.py`	Passe-plat moderne, remplissage, migration historique.	4
	Total	**37**

Table 18.1 – Couverture fonctionnelle des tests.

18.2 Qualité du code

Le projet est configuré pour `ruff`, à la fois pour le lint et pour le formatage :

```
ruff check .          # lint
ruff format .         # formatage
pytest                # tests
```

Listing 18.2 – Vérifications habituelles.

Les règles activées couvrent les erreurs (`E`, `F`), les avertissements (`W`), le tri des imports (`I`), les pièges fréquents (`B`), la modernisation des annotations (`UP`) et les simplifications (`C4`). La longueur de ligne cible est de 100 caractères.

18.3 Construire l'exécutable

Le script `build_exe.ps1` et le fichier `IcareWeb.spec` emballent l'application et ses ressources (gabarits, statiques, séquences) dans un exécutable unique.

```
.\build_exe.ps1     # produit dist\IcareWeb.exe
```

Listing 18.3 – Construction.

18.4 Sources et archive

Pour ne pas embarquer de fichiers inutiles, `.gitignore` exclut les caches Python (`__pycache__`, `*.pyc`), les environnements virtuels, les artefacts PyInstaller (`build/`, `dist/`, `*.spec`) et les fichiers produits à l'exécution (`icare.log`, `icare_settings.json`, `Palettes/`).

À noter

Le dossier `Palettes/` est ignoré par Git car il contient des données locales, mais il peut être sauvegardé à part si vous voulez conserver vos couleurs favorites.

Cinquième partie

Des lumières pilotées par Ableton

Chapitre 19

La refonte multi-projecteurs

Le contrôleur d'origine ne pilotait qu'un seul projecteur. Pour produire de vrais jeux de lumière, Icare gère désormais un **patch** complet : une liste de projecteurs, chacun décrit par un *profil* (plan de canaux) et une *adresse* de départ dans l'univers DMX.

19.1 Les profils de projecteurs

Un profil dit quel canal fait quoi. Icare embarqué les profils suivants, qui correspondent au matériel :

Profil	Canaux	Faisceau	Détail
`rgbwa_par_5ch`	5	30ř	Rouge, Vert, Bleu, Blanc, Ambre.
`wild_wash_132_6ch`	6	75ř	Intensité, Strobe, Rouge, Vert, Bleu, sensibilité son.
`cameo_qspot40_9ch`	9	15ř	Intensité, intensité fine, Strobe, R, G, B, Blanc, macro, température.
`stairville_mfog1500_3ch`	3	—	Volume brouillard, intervalle, durée.

Table 19.1 – Profils livrés (l'angle de faisceau alimente la simulation 3D).

19.2 Le patch par défaut

Par défaut, les appareils sont posés à la suite à partir du canal 1 :

Appareil	Profil	Canal de départ	Canaux
ParLED1 …ParLED6	RGBWA 5 canaux	1, 6, 11, 16, 21, 26	5
Wash1, Wash2	Wild Wash 132	31, 37	6
QSpot1, QSpot2	Cameo Q-Spot 40	43, 52	9
Fog	M-Fog 1500	61	3

Table 19.2 – Patch par défaut (63 canaux utilisés).

Le patch est validé au chargement : un profil inconnu, un chevauchement de canaux ou une adresse hors de l'univers sont refusés. Il vit dans `patch.json` à côté de l'exécutable et peut être modifié depuis l'API `POST /api/patch`.

19.3 Comment le rendu gère les différences

Chaque mode « génératif » (couleur fixe, effets, aléatoire, audio) produit une couleur et une intensité globales. Le moteur l'écrit sur chaque projecteur :

— si le profil possède un **canal d'intensité**, il est écrit tel quel et les couleurs en brut ;
— sinon (PAR RGBWA 5 canaux), les couleurs sont **multipliées par l'intensité** : c'est l'intensité qui pilote la luminosité.

Le brouillard possède ses propres canaux, indépendants des modes : on peut l'activer séparément via `POST /api/fog`.

À noter

Le Q-Spot 40 et le Wild Wash sont des projecteurs **fixes** : leurs profils n'ont ni pan ni tilt. Les curseurs de position de l'onglet Direct ne servent donc qu'à la visualisation 3D.

Chapitre 20

Les « looks »

Un **look** est un état de lumière statique que l'on peut rappeler : une liste de cibles et, pour chacune, une couleur et une intensité.

20.1 Cibles

Une cible peut être :

— `"*"` : tous les projecteurs ;
— le nom d'un **groupe** (`pars`, `wash`, `spots`, `fx`) ;
— le nom d'un **projecteur** précis (`QSpot1`, `Fog`…).

Les cibles sont fusionnées dans l'ordre `"*"`, puis le groupe, puis le nom : on met donc une base globale et on surcharge au besoin.

20.2 Résolution

`resolve()` transforme le look en valeurs de canaux concrètes. Pour un projecteur sans canal d'intensité, l'intensité demandée « multiplie » les couleurs. Exemple de look :

```
{
  "name": "Refrain",
  "targets": {
    "pars":  { "r": 0, "g": 0, "b": 255, "dimmer": 220 },
    "QSpot1": { "r": 255, "g": 80, "b": 0 }
  },
  "fog": { "level": 120 }
}
```

Listing 20.1 – Un look : base bleue, projecteurs rouges.

Les looks sont de simples fichiers JSON dans `Looks/`, créés depuis l'onglet **Show** : on choisit un nom, une cible, une couleur et une intensité, puis on sauvegarde.

Chapitre 21

Le lien avec Ableton Live

21.1 Principe

ICARE écoute Ableton Live par le réseau grâce au script distant gratuit AbletonOSC. Aucun câble supplémentaire : tout passe en OSC sur `127.0.0.1`.

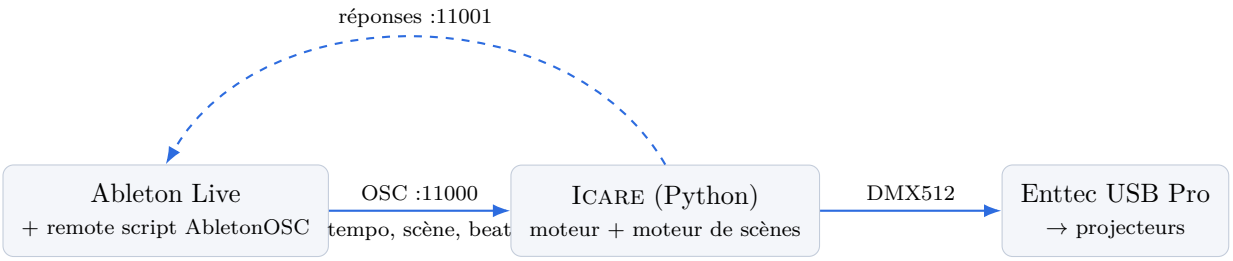


FIGURE 21.1 – Boucle Ableton → ICARE → DMX.

Ce qu'ICARE observe en continu :

Adresse OSC	Information
`/live/song/get/tempo`	Tempo (BPM).
`/live/song/get/is_playing`	Lecture en cours.
`/live/view/get/selected_scene`	Scène sélectionnée.
`/live/scene/get/name`	Nom de chaque scène.
`/live/song/get/beat`	Compteur de temps (beat events).

TABLE 21.1 – Les principales données suivies.

21.2 Installation

1. Exécuter `tools\install_abletonosc.ps1` : le script télécharge AbletonOSC et le copie dans `Documents\Ableton\User Library\Remote Scripts`.
2. Redémarrer Live.
3. *Préférences* → *Link, Tempo & MIDI* → Control Surface = **AbletonOSC**.
4. Dans ICARE, onglet **Show** → **Connecter Ableton**. Le témoin passe au vert.

À noter

Sans Ableton, le lien reste simplement « déconnecté » : Icare continue de fonctionner normalement.

21.3 Mapper les scènes aux looks

L'onglet **Show** liste les scènes d'Ableton (récupérées via OSC). Pour chacune, on choisit le look à déclencher. Dès qu'une scène est sélectionnée ou lancée dans Ableton, Icare applique le look correspondant.

```
{ "enabled": true,
  "mapping": { "INTRO": "intro_bleu", "A": "couplet", "CHORUS": "refrain" } }
```

Listing 21.1 – Mapping persistant (scenes.json).

21.4 Produire son show

Une méthode simple, adaptée au template Ableton du studio :

1. Dans Icare, créer un look par ambiance : `INTRO`, `A`, `B`, `CHORUS`, `BREAK`, `SOLO`, `OUTRO`.
2. Mapper chaque scène Ableton au look du même nom, une fois pour toutes.
3. En répétition ou en concert, il suffit de lancer les scènes dans Ableton : les lumières suivent.
4. Réglage fin : le *beat* d'Ableton pourra servir plus tard à caler les effets (strobe sur le temps, chenillard à la mesure).

Chapitre 22

La simulation 3D

L'onglet **Show** dessine toute la scène, sans aucun matériel, à partir de l'état DMX réel (`GET /api/simulation`). On peut ainsi concevoir les looks et vérifier le rendu avant la répétition.

— chaque projecteur est placé à sa position `(x, z)` et orienté par son `angle` ;
— son **faisceau** a le rayon réel de son profil : $r = d \times \tan(\text{angle}/2)$ — un PAR (30ř) est étroit, un Wild Wash (75ř) très large, un Q-Spot (15ř) serré ;
— la couleur et l'intensité du faisceau suivent les canaux DMX.

Un rendu optique, pas un dessin. Les faisceaux ne sont pas des cônes pleins : ils sont calculés par un shader qui reproduit les propriétés d'un faisceau réel.

— **Décroissance radiale** : le faisceau est plus intense sur son axe et s'éteint doucement vers ses bords (pas de bord net) ; un Q-Spot a un cœur serré, un Wild Wash est large et homogène.
— **Décroissance axiale** : l'intensité diminue avec la distance.
— **Pièce au sol** : à l'intersection du faisceau et du sol, une *flaque* de lumière elliptique apparaît, dimensionnée par l'angle.
— **Tone mapping ACES** et sortie sRGB : les recouvrements de faisceaux «brûlent» vers le blanc au lieu de saturer brutalement, comme sur une image filmée.
— La salle est **sombre** ; seul ce que la lumière touche est visible. Un curseur *Exposition* règle l'ensemble.

La fumée est éclairée par les lumières. La fumée n'est pas auto-éclairée : chaque particule reçoit la lumière *qui l'atteint réellement*. On teste, pour chaque faisceau allumé, si la particule tombe dans son cône, puis on accumule contribution radiale et axiale. La couleur et l'opacité de la particule en découlent :

$$\text{éclairement} = \sum_{\text{feux}} I_f \cdot \left(1 - \frac{r}{r_{\max}}\right)^2 \cdot \left(0{,}35 + 0{,}65\left(1 - \frac{d}{L}\right)\right).$$

Hors de tout faisceau, il ne reste qu'un **très léger** voile naturel ($\approx 3{,}5\,\%$), conformément au rendu réel : sans lumière qui la frappe, la fumée est sombre.

Chapitre 23

Architecture logicielle mise à jour

Module	Rôle
`webapp/fixtures.py`	Profils de projecteurs et patch.
`webapp/looks.py`	Modèle de look, résolution, stockage.
`webapp/osc.py`	Encodeur/décodeur OSC minimal.
`webapp/ableton.py`	Client AbletonOSC + moteur de scènes.
`webapp/dmx_controller.py`	Moteur multi-projecteurs (thread de sortie).
`webapp/api_show.py`	API patch / looks / scènes / Ableton.
`webapp/templates/show.html`	Onglet Show.

TABLE 23.1 – Nouveaux modules de la refonte.

Le moteur DMX garde la même surface publique qu'avant (`set_values`, `strobe`, `effect_*`, `audio_push`, `health`, ...), mais les applique désormais au **patch entier**. Les anciennes fonctions de l'onglet Direct et le lecteur de séquences continuent donc de fonctionner.

À noter

Le contrôleur par défaut écrivait une seule adresse de couleur. Il écrit maintenant une *frame* complète : pour chaque projecteur, les canaux de son profil sont remplis, ce qui rend possible un rendu différent d'un appareil à l'autre.

Annexe A

Aide-mémoire du DSL

Jeton	Exemple	Effet
`""`	`""`	Ne change rien ce temps-là.
`|`	`|`	Continue l'effet multi-temps précédent.
`uni_n_R_G_B`	`uni_4_255_0_0`	Rouge fixe pendant 4 temps.
`fond_n_c1_c2`	`fond_8_255_0_0_0_0_255`	Fondu rouge → bleu sur 8 temps.
`str_n_R_G_B_f`	`str_2_255_255_0_120`	Strobe jaune pendant 2 temps à 120 Hz.
`str_n_R_G_B`	`str_3_0_0_255`	Strobe bleu, fréquence par défaut (60 Hz).

TABLE A.1: Tous les jetons acceptés.

Règles. Une composante de couleur doit être comprise entre 0 et 255, sans quoi le jeton est rejeté. La longueur `n` est au minimum de 1. Un préfixe inconnu, un mauvais nombre d'arguments ou un texte non numérique produisent une `DSLError`, journalisée et ignorée pendant la lecture Live.

Fondu sur plusieurs lignes. On écrit le jeton complet sur la première ligne, puis un `|` sur chacune des lignes suivantes : le nombre de `|` doit valoir `n-1` pour couvrir exactement la durée.

```
Ligne 1 :  fond_4_255_0_0_0_0_255
Ligne 2 :  |
Ligne 3 :  |
Ligne 4 :  |
```

Listing A.1 – Fondu de 4 temps (1 ligne d'en-tête + 3 continuations).

Annexe B

Dépannage

Le témoin DMX reste rouge / «DMX HS.
Le serveur ne répond pas : vérifiez qu'Icare tourne encore et que l'adresse est bien `127.0.0.1:5000`.

La connexion échoue avec «No USB-DMX adapter found.
Aucun port COM ne contient «USB» dans sa description. Saisissez le port manuellement (par ex. `COM3`) dans le champ *Port*. Vérifiez aussi que le pilote de l'adaptateur est installé.

Les projecteurs ne réagissent pas.
Contrôlez deux points : le *canal de base* doit correspondre à celui programmé sur le projecteur, et la ligne DMX doit être correctement chaînée (adaptateur, projecteurs, bouchon de fin si nécessaire).

L'audio-réactif ne fonctionne pas.
Le navigateur doit avoir l'autorisation microphone. L'API `getUserMedia` n'existe que sur les pages servies en `localhost` ou en HTTPS : Icare étant local, c'est le cas.

Le tempo mesuré est bizarre.
Les appuis trop espacés (plus de 3 secondes d'écart) sont ignorés. Utilisez **Reset** puis tapez de nouveau, plus régulièrement.

J'ai un ancien fichier de séquence.
Rien à faire : le format historique est migré automatiquement à l'ouverture.

Annexe C

Glossaire

Blueprints
Mécanisme Flask qui regroupe des routes par thème (`api_live`, `api_sequence`, ...).

Canal de base
Premier canal occupé par un projecteur.

DMX512
Standard de pilotage de lumières : 512 canaux de 0 à 255 sur un bus RS-485 à 250 kbauds.

DSL
Domain Specific Language : ici, le petit langage de chaînes qui décrit l'action d'un module sur un temps.

Enttec USB Pro
Adaptateur USB-DMX très répandu ; son protocole est celui implémenté par `pydmxIcare.py`.

Fixture
Un projecteur DMX.

Frame / image
L'ensemble des 512 valeurs envoyées sur la ligne.

HSV
Représentation d'une couleur par teinte, saturation et valeur ; pratique pour générer des arc-en-ciels.

Overlay
Effet appliqué par-dessus le mode courant (ici, le strobe).

RMS
Root Mean Square : mesure d'énergie d'un signal sonore.

Séquence
Fichier JSON décrivant un show ligne par ligne.

Thread
Fil d'exécution. Icare en utilise deux : la sortie DMX et la lecture Live.

Univers
Les 512 canaux d'une ligne DMX.